

Topic 6.3

6.3: Derived Classes

Consider the following program fragment:

```
class Base
{
public:
    Base(void);
    void print(void);
    void read();
    void apply();

protected:
    int    internal_data1;
    double internal_data2;
};

class Derived : public Base
{
public:
    Derived(void);
    void print(void);
    void read();

protected:
    char internal_data3[24];
};
```

Class `Base` is known as the *base class*.

Class `Derived` is known as the *derived class* (it can be the base for further derived classes).

The declaration `class Derived : public Base` means this is a *public* derivation. Other types are possible, and this will be explained later.

Remember that *private* members are only accessible

- from within member functions of the same class; or
- from within friend functions; or
- from within member functions of friend classes.

In our discussion on classes, I introduce here the new scope *protected*, which relaxes the visibility restrictions of *private*, and allows access by all member functions and friend functions of classes derived from this class.

I have previously talked about *overloaded* operators and names, in which case the functions have different *signatures*, i.e. a different number and/or different types of parameters. In this fragment, the member functions `print()` and `read()` of the `Base` class are *overridden* in the `Derived` class. Their signatures are the same.

If a base class defines multiple versions of some member function (each with a unique signature), and a derived class overrides only one of them, the remaining functions will not be visible to objects of the derived class. I will illustrate this with a simple example in Derived Classes and Operator in Topic 5 of this module.

Topic 6.4 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-6-dot-4>)